



UF1

Servicios de red implicados en el despliegue

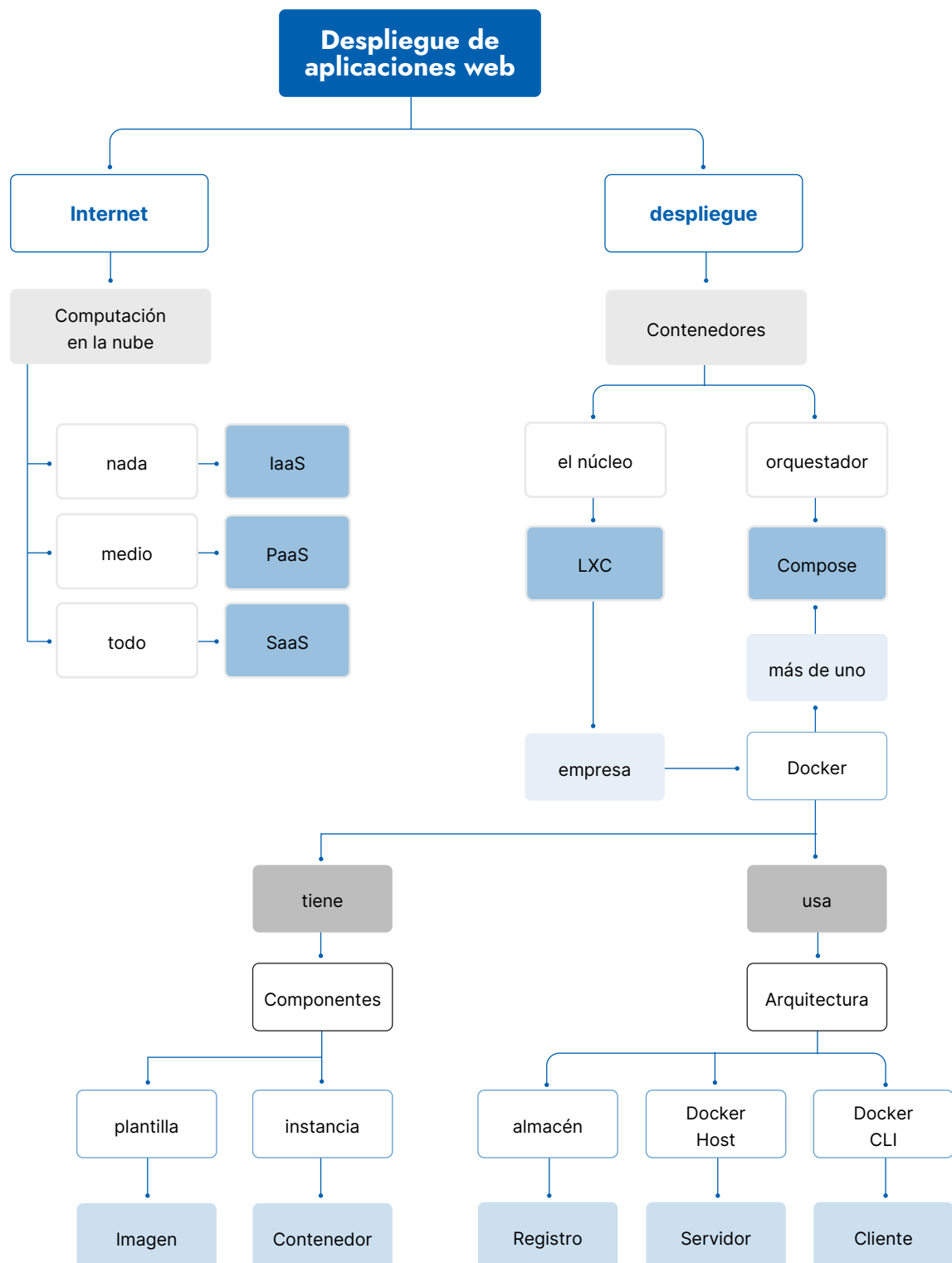
Despliegue de aplicaciones web

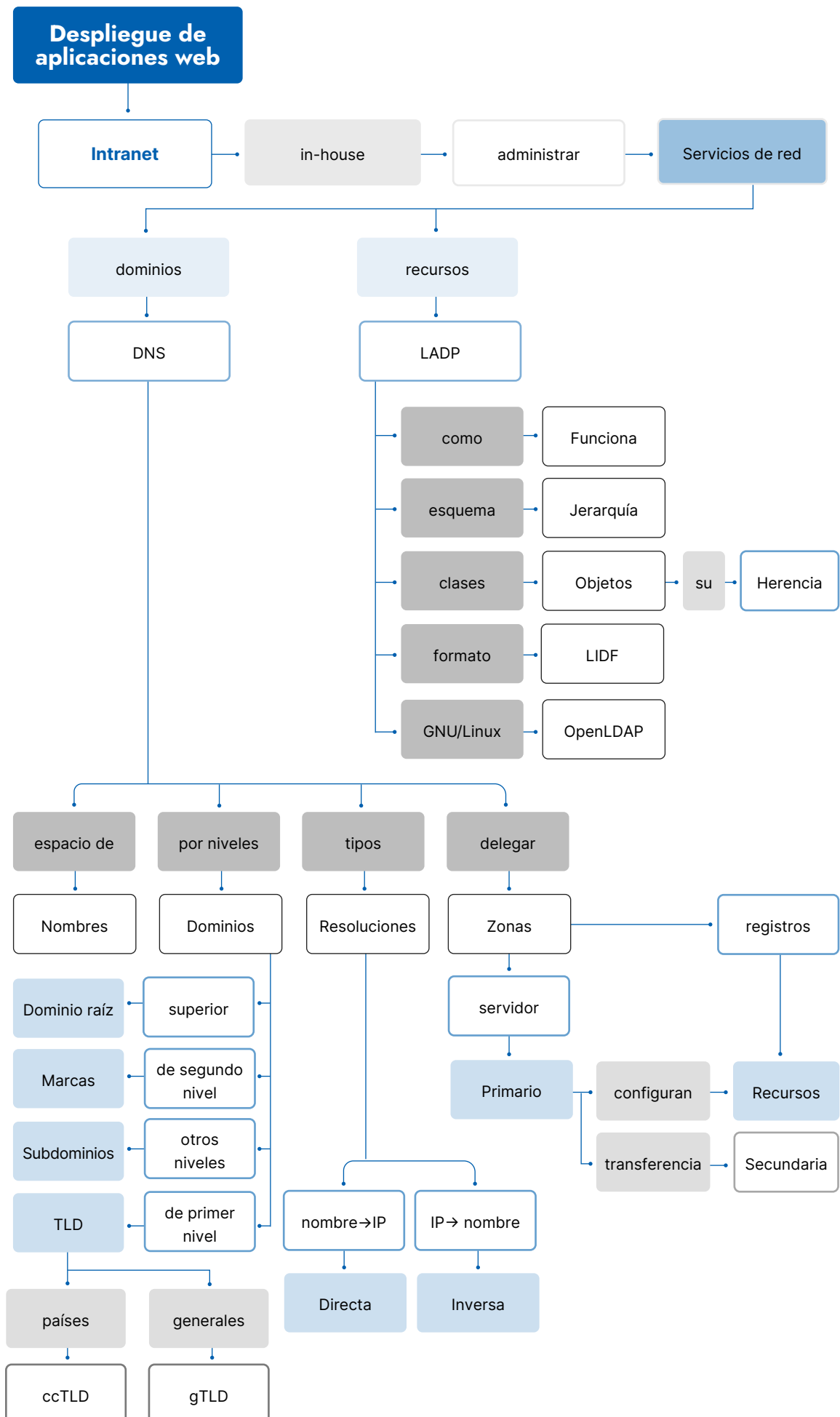
ÍNDICE

Mapa conceptual	04
1. Despliegue de aplicaciones web	06
1.1. Introducción al despliegue de aplicaciones web.....	06
1.2. Alojamiento interno (<i>in-house</i>)	08
1.3. Computación en la nube.....	09
A. Infraestructura como servicio (IaaS).....	09
B. Plataforma como servicio (PaaS)	10
C. Software como servicio (SaaS).....	10
2. Sistema de nombres de dominio (DNS).....	12
2.1. Espacio de nombres de dominio.....	12
2.2. Dominios por niveles	13
A. Dominio raíz.....	13
B. Dominios de primer nivel.....	13
C. Dominios de segundo nivel.....	14
D. Subdominios.....	14
2.3. Tipos de dominio	15
2.4. Resoluciones directas y resoluciones inversas	16
2.5. Zonas primarias y secundarias.....	17
2.6. Tipos de servidores DNS	18
2.7. Registro de recursos	18
A. Tipos de registros de recursos.....	19
3. Protocolo ligero de acceso a directorios (LDAP)	21
3.1. LDAP.....	22

A. Funcionamiento.....	22
B. Esquemas de jerarquía	23
C. Clases de objeto (<i>objectclass</i>)	24
D. Herencia	24
E. Esquemas (<i>schemas</i>).....	25
F. Formato de intercambio de datos LDAP (LDIF)	25
G. Ventajas.....	26
3.2. OpenLDAP	26
4. Despliegue con contenedores	28
4.1. Contenedores	29
A. LXC	29
B. Docker	30
4.2. Orquestadores	33
A. Docker compose	33

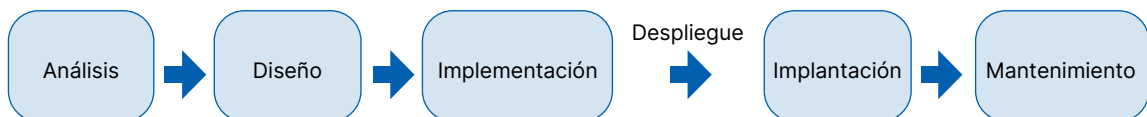
Mapa conceptual





01 Despliegue de aplicaciones web

Vamos a empezar aprendiendo qué significa el concepto de **despliegue** (*deployment*). Para ello, vamos a recordar en la figura siguiente el ciclo de vida clásico de un desarrollo de software, ya que las aplicaciones web requieren ser desplegadas. Serán las tareas que realizaremos para finalizar la fase de implementación o desarrollo y trasladar nuestra aplicación web probada y terminada a la siguiente fase de implantación o producción. Para ello, hemos de instalar y configurar los diferentes servicios de red o web que se vayan a utilizar por parte de nuestra aplicación web.



Esquema 1. Ciclo de vida de un desarrollo de software.

Este despliegue de software es una tarea repetitiva con muchas acciones por realizar que, si no están automatizadas, pueden llevarnos a introducir errores humanos. Si no se actualizan todas las dependencias o copiamos los archivos de configuración adecuadamente en el despliegue de forma precisa, la nueva aplicación web desplegada no llegará a funcionar correctamente. Por lo tanto, nuestro objetivo será automatizarlos lo máximo posible, sin olvidar efectuar una revisión final por nuestra parte de que todo está funcionando correctamente.

Pero primero vamos a ver cuáles son las características de una aplicación web para desplegar.

1.1. Introducción al despliegue de aplicaciones web

Una aplicación web va a ejecutarse sobre uno o varios servidores web. Inicialmente, estos eran físicos o reales, es decir, utilizaban todo el equipo para esta tarea. En la actualidad, estos equipos van a ser virtualizados completamente o se utilizarán contenedores para consumir los mínimos recursos posibles.

Una de las peculiaridades de las aplicaciones web es que vamos a querer dar servicio concurrente a todos nuestros usuarios, que pueden ser miles o millones, en cuyo caso deberemos trabajar con sistemas distribuidos que formen un clúster de equipos coordinados, lo que aumenta la complejidad del despliegue.

Por eso también deberemos enfrentarnos a decisiones de escalabilidad del sistema decidiendo que su crecimiento sea vertical (aumentando los recursos del servidor con más RAM, más capacidad de disco o más *cores*, o núcleos de microprocesadores) u horizontal (aumentando el número de servidores iguales) para producir mayor tolerancia a fallos y poder equilibrar la carga entre todos ellos. Esto se ve gráficamente en la siguiente figura.

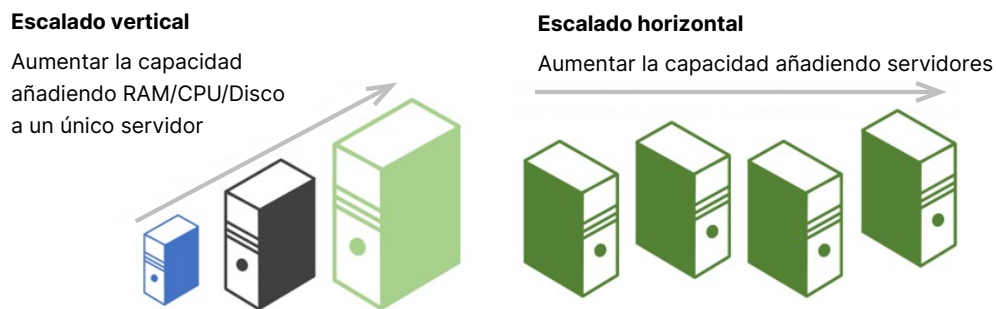


Figura 1. Tipos de escalado.

De esta manera, tendremos que dominar:

- » Los diferentes servicios de red que vayamos a integrar o utilizar con nuestra aplicación web.
- » El ciclo de vida de una imagen de máquina virtual denominada AMI y su automatización.
- » Optimización de costes en despliegue de instancias y su monitorización.
- » El despliegue de infraestructura de máquinas virtuales puede ser en hostings tradicionales en nubes públicas o privadas o en los nuevos proveedores de computación en la nube.

	INTERNAMENTE	IAAS	PAAS	SAAS
	Aplicaciones	Aplicaciones	Aplicaciones	Aplicaciones
	Datos	Datos	Datos	Datos
	Tiempo de ejecución	Tiempo de ejecución	Tiempo de ejecución	Tiempo de ejecución
	Middleware	Middleware	Middleware	Middleware
	SO	SO	SO	SO
	Virtualización	Virtualización	Virtualización	Virtualización
	Servidores	Servidores	Servidores	Servidores
	Almacenamiento	Almacenamiento	Almacenamiento	Almacenamiento
	Equipos de red	Equipos de red	Equipos de red	Equipos de red

Gestionado por tu equipo

Gestionado por tu proveedor

Tabla 1. Despliegue interno versus IaaS, PaaS y SaaS.

1.2. Alojamiento interno (*in-house*)

El alojamiento interno (*in-house*) es el modelo clásico en el que una organización o empresa aloja todos sus servidores, aplicaciones y servicios dentro de sus instalaciones. Esto implica instalar y mantener todos los recursos de hardware necesarios. Por ejemplo:

Servicio DNS	Para que todos nuestros equipos puedan utilizar sus nombres de dominios y poder traducirlos a las IP privadas de nuestra intranet.
Servicio de directorio activo	Con validación de usuarios centralizada con SAMBA o LDAP/ OpenLDAP para que nuestros usuarios puedan usar todas las aplicaciones web de nuestra intranet.
Servicios web generalistas	O específicos de tecnologías como Java para las aplicaciones web propias o de terceros.
Servicios de transferencia de archivos (FTP)	Que nos permiten subir o bajar archivos grandes o muchos pequeños rápidamente.
Servicios de correo electrónico.	Con los diferentes protocolos de envío (SMTP) o de recepción (POP o IMAP).
Servicios de conexión remota	Por terminal (SSH) o por GUI (VNC, RDP, etc.).

Esquema 2.

Podríamos tener un campus de formación con Moodle, un CMS con WordPress o nuestras propias aplicaciones web.

Se puede considerar que tendríamos una nube privada con servidores de virtualización, como podría ser ProXmox VE u otros para dar soporte a todos estos servicios con máquinas o contenedores virtualizados. Esta opción cada vez se utiliza menos, ya que tiene un elevado coste de implantación y mantenimiento, además de requerir una infraestructura de sistemas de respaldo en caso de caída de Internet o de la red eléctrica. Por ello, la tendencia actual es cada vez más trasladar total o parcialmente esta infraestructura propia a una nube pública para que la computación en la nube se encargue de administrar parte de la infraestructura que vayamos a contratar.

1.3. Computación en la nube

En 2006, surgió el concepto de “computación en la nube” (*cloud computing*). Fue la división de servicios web de Amazon (AWS) la primera en ofrecer un servicio de virtualización de servidores a gran escala llamado EC2 (*Elastic Compute Cloud*) y otro de almacenamiento masivo de objetos llamado S3 (*Simple Storage Service*) con el objetivo de ofrecer a los clientes con CPD en propiedad un servicio similar, pero virtualizado y en pago por uso, con las ventajas de escalabilidad, ausencia de inversión inicial y menor coste. En la siguiente figura se puede ver el panorama general clasificado de los servicios de computación de AWS.

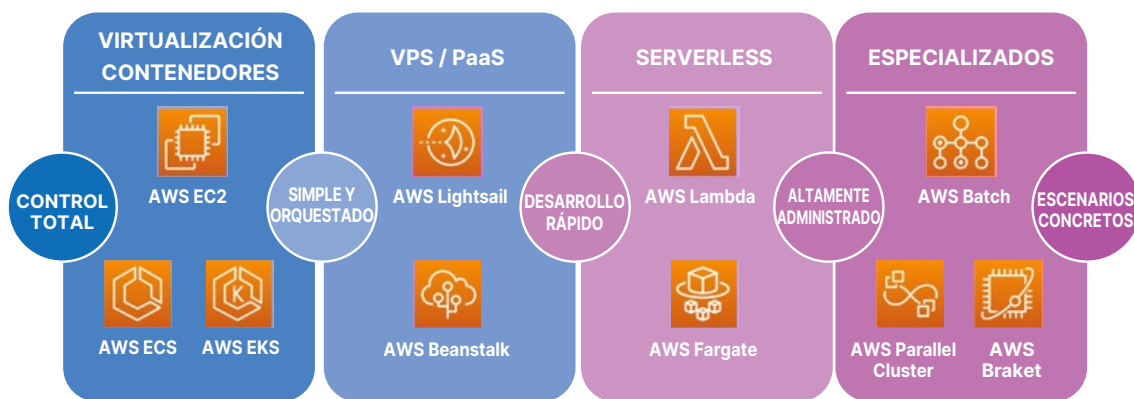


Figura 2. Servicios de AWS.

Desde entonces, el uso extendido de aplicaciones web y de móviles, servicios multimedia masivos —como los servicios de *streaming* por suscripción de películas—, series o música y el procesamiento de datos continuo relacionado con el *Big Data* han experimentado una gran evolución en los servicios de computación, más allá de ofrecer máquinas virtuales que sustituyan a las propias de un CPD.

Veamos en qué partes se dividen los servicios de computación en la nube donde deberemos desplegar nuestras aplicaciones dependiendo de qué queramos encargarnos, desde administrar todo el equipo (IaaS) a solo encargarnos de nuestra aplicación web (SaaS) o contar con una opción intermedia para desarrollar también desde la nube (PaaS). Veamos cada una de ellas.

A. Infraestructura como servicio (IaaS)

Algunos de los planes de hosting tradicionales funcionan sobre este nivel, por ejemplo, el *cloud hosting* o el de servicio dedicado que puede estar contratado en cualquiera de los IaaS que existen hoy en día.

La diferencia es que, en el *cloud hosting*, el proveedor del servicio nos facilita una consola de administración que nos ayuda en la gestión del dominio y de nuestras webs, mientras que, en el servicio dedicado, se nos dará acceso a una máquina sin ninguna ayuda y nosotros seremos los que nos encargaremos de hacer todo lo que queramos, por lo que tendremos que ser unos buenos administradores de sistemas y tendremos que dominar el mundo de la computación en la nube. Los tres principales proveedores de infraestructura como servicio (IaaS) son AWS, Azure y Google.

B. Plataforma como servicio (PaaS)

En este modelo, el desarrollador se olvida del hardware y del sistema operativo y se preocupa solo de la aplicación y los datos por desplegar. Típicamente, se utiliza para desplegar aplicaciones web, bases de datos y *middleware*. Servicios como el de la arquitectura Heroku (en la siguiente figura) nos facilitarán el despliegue de nuestras aplicaciones web, que podremos subir a este servicio en la nube. Desde su panel de control (*dashboard*) o utilizando su CLI, podremos desplegar, probar y, finalmente, publicar nuestras aplicaciones web sobre el servicio SaaS final donde vayamos a explotarnos.

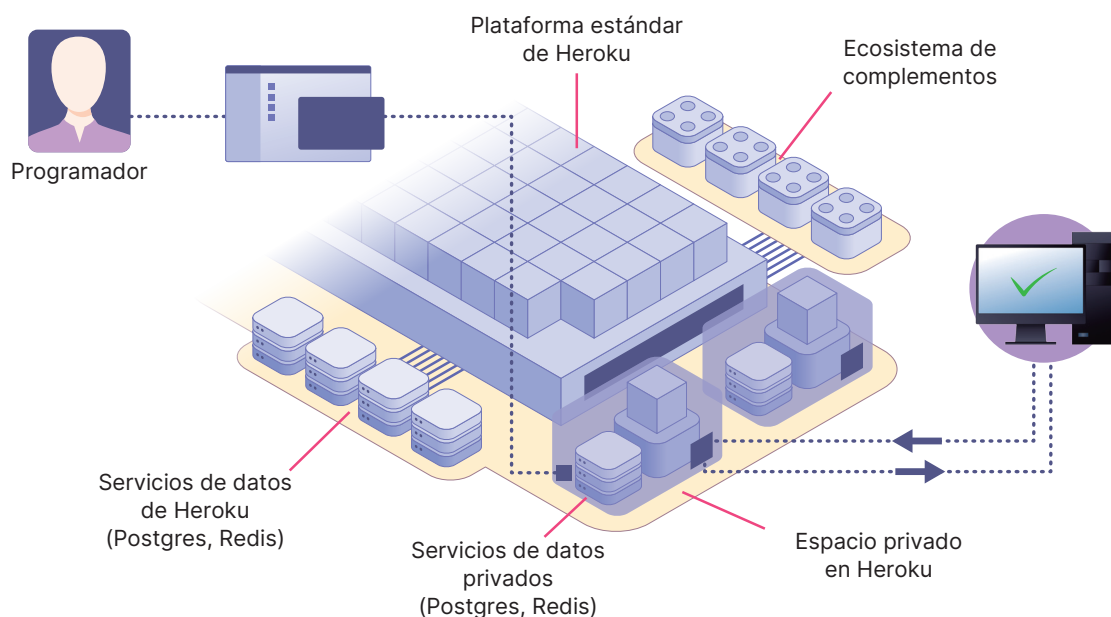


Figura 3. Arquitectura Heroku.

C. Software como servicio (SaaS)

En esta tipología, el proveedor del servicio en la nube nos da soporte a todos los niveles, incluso sobre la aplicación que vamos a utilizar con él. El usuario se limita a usar la aplicación contratada y se olvida de lo demás. Es lo opuesto a tenerlo en nuestras instalaciones (*in-house*), donde nos encargamos de gestionar todo nosotros.

Podemos utilizarlo sin tener que saber programar. Por ejemplo, contratando el sistema gestor de contenidos (CMS) WordPress, el planificador de recursos de empresa (ERP) Odoo o el software de comercio electrónico (*e-commerce*) PrestaShop. Para ello, solo tendremos que contratar alguno de estos servicios con una de las muchas empresas de hosting. Lo mejor es que conozcamos las diferentes opciones que podemos contratar y el coste de cada una de ellas.

1

CASO PRÁCTICO



¿CÓMO FUNCIONA EL MODELO DEVOPS?

Tradicionalmente, los departamentos de desarrollo y los de operaciones o sistemas funcionan de manera independiente en las organizaciones. Los primeros se centran en la fase de implementación y los siguientes, en la fase de implantación. Pero, con la aparición de las aplicaciones web y la necesidad de desplegarlas, ha surgido este nuevo modelo llamado DevOps. ¿En qué mejora este modelo el funcionamiento de dichos departamentos?

SOLUCIÓN

El hecho de que las aplicaciones web estén en versión beta continua significa que hay aplicaciones con mejoras o versiones nuevas casi a diario con millones de usuarios activos todos los días. La utilización del nuevo modelo DevOps fusiona ambos departamentos para que todo el equipo trabaje con metodologías ágiles que le permitan estar implicado en todas las fases o tareas, desde el desarrollo de una mejora, pasando por su validación y su verificación mediante pruebas, hasta la creación de un nuevo lanzamiento (*release*) que se desplegará para volver a estar en producción. Y todo esto, aplicando metodologías CI/CD, es decir, de integración y distribución continuas.

Puedes ver más información en <https://aws.amazon.com/es/devops/what-is-devops>



VÍDEO LAB

Para consultar el siguiente vídeo sobre autenticación de aplicaciones, escanea el código QR o [pulsa aquí](#).

02 Sistema de nombres de dominio (DNS)

El sistema de nombres de dominio, DNS (*Domain Name System*), se centra en la administración de los nombres de dominio, dando el servicio de resolución o traducción a su correspondiente IP. Se realiza de forma distribuida entre todos los servidores DNS de la red de redes basada en TCP/IP.

Recordemos que en una red basada en la familia de protocolos TCP/IP las máquinas se identifican mediante su dirección de red o número IP (versión 4 o 6). Sin embargo, para las personas resulta mucho más sencillo recordar un nombre simbólico de una marca o un producto que asocia un dominio a una máquina concreta. Y también es más fiable, ya que la dirección IP puede cambiar más habitualmente, mientras que, con el nombre de la organización, es menos probable que cambien, aunque también puede pasar. Ello hace necesario establecer un mecanismo de traducción de nombres de máquina de los dominios a direcciones IP únicas para localizar el servidor donde reside un sitio web, que es el servicio DNS (*Domain Name System*). Un dominio o nombre de dominio es el nombre que identifica a un sitio web. El dominio tiene que ser único en Internet. Por ejemplo, www.mheducation.es es el nombre de dominio del sitio web de McGraw-Hill en España.

2.1. Espacio de nombres de dominio

El servicio DNS se compone de una base de datos distribuida (en varias máquinas conectadas en red a través de Internet) en la que se almacenan las asociaciones de nombres de dominios y sus direcciones IP. Esta base de datos distribuida está clasificada por nombres de dominio donde cada uno de ellos es una rama en un árbol invertido llamado **espacio de nombres de dominio**.

El árbol de nombres de dominio comienza en un nodo denominado raíz que está situado en la cima, es decir, en el nivel superior. Por debajo de él, puede existir un número indeterminado de nodos de nivel inferior (hasta un máximo de 127). Normalmente, se utilizan hasta cinco o seis niveles; utilizar más ya puede dificultar la memorización de dicho nombre simbólico. Por ejemplo, <http://www.educacionyfp.gob.es/> tiene tres niveles, sin embargo,

<https://incual.educacion.gob.es/> tiene cuatro (recuerda que el nodo raíz va implícito a la derecha y no se muestra dicho punto final). La primera pertenece al dominio principal del Ministerio de Educación, mientras que el segundo es un subdominio del Instituto Nacional de las Cualificaciones.

Los nodos o elementos del árbol (cada elemento del nombre de dominio) se identifican o etiquetan mediante nombres “no nulos” que pueden contener un determinado número de caracteres cada uno de ellos (máximo, 63). El nodo raíz, el superior que se omite a la derecha del nombre, se identifica mediante un nombre “nulo” (de 0 caracteres, el mínimo). El nombre completo de un nodo está formado por el conjunto de nombres que forman la trayectoria desde ese nodo (más a la izquierda) hasta el nodo raíz. Como separador de nombres, se utiliza el carácter punto (.).

Cada elemento o nodo se identifica de forma única dentro de la jerarquía que genera el árbol de nombres de dominio. Este nombre de dominio completo se llama nombre de dominio completo o totalmente calificado (FQDN, del inglés *Fully Qualified Domain Name*). Cada nivel va delegando autoridad en su nivel inferior tantas veces como se permita dicha delegación.

2.2. Dominios por niveles

El espacio de nombres DNS está basado en una estructura jerárquica de árbol invertido con distintos niveles de dominio. Podemos crear hasta 127, pero lo habitual es tener un máximo de tres, cuatro o cinco, es decir, uno, dos o tres subdominios propios del dominio principal. Respecto de esa estructura jerárquica, existen los siguientes niveles de dominios:

A. Dominio raíz

De este dominio cuelga toda la estructura del espacio de nombres DNS. Se simboliza con un punto. Bajo el directorio raíz, hay dominios de primer nivel. El dominio raíz se encarga de gestionar la información sobre los dominios de primer nivel, en concreto, sobre los nombres de esos dominios y sobre los servidores encargados de la gestión de esos dominios. El organismo que se encarga de su gestión es el ICANN (*Internet Corporation for Assigned Names and Numbers*), encargado global de que todo funcione correctamente y de aprobar actualizaciones.

B. Dominios de primer nivel

Son dominios que, en la estructura del espacio de nombres DNS, se encuentran bajo el dominio raíz. A un dominio de este tipo también se le llama TLD (*Top-Level Domain*). Dominios de este tipo son, por ejemplo, los de tres letras, como .com, .org o .net, entre otros, o los de dos letras, de países como .es, .is o .aq, entre otros.

En algunos países se combinan ambas, por lo que podemos encontrar .gob.es, que ya serían del siguiente nivel. Bajo los TLD, están los dominios de segundo nivel. De la gestión de cada dominio de primer nivel se encarga una determinada organización, que usa sus servidores DNS.

Todos los dominios de este nivel tienen su centro de información de red propio (NIC, *Network Information Center*). De la gestión del dominio .es, que existe desde 1988, se encarga ESNIC (www.nic.es) que, actualmente, delega en **dominios.es**, perteneciente al ente Red.es, que depende del Ministerio de Asuntos Económicos y Transformación Digital de nuestro país.

Los servidores DNS de este nivel contienen información relativa a los servidores de dominio de segundo nivel dependientes del dominio. Por ejemplo, cada servidor del dominio .es tiene información sobre todos los servidores de su dominio, como el de mheducation.es o el de gva.es, que ya son de segundo nivel.

C. Dominios de segundo nivel

Son los dominios que se encuentran bajo los TLD. Cada uno de estos dominios está registrado a favor de una determinada entidad (empresa, universidad, órgano, persona, etc.). La entidad propietaria del dominio es la encargada de la gestión de su dominio y los subdominios que haya decidido crear. Para un dominio de este tipo, se tienen uno o varios servidores DNS que tienen información sobre máquinas disponibles en el dominio, sobre posibles subdominios y sobre servidores DNS del dominio y de sus subdominios. Una entidad que desee disponer de un dominio, debe registrarlo ante un registrador oficial autorizado por ICANN. En el caso de los dominios .es, es este ente nacional quien autoriza a los registradores que lo podrán comercializar.

D. Subdominios

Es el último tipo de nivel, forman parte de un dominio de segundo nivel. De forma recurrente, podemos tener varios subdominios debajo de cada uno. Al ser parte de un dominio privado de una organización, no hace falta que se registre como un dominio de segundo nivel. Si puede crearse su zona delegada o privada (lo veremos más adelante), cada organización podrá crear o no dichos subdominios. Si no, deberá gestionarlo a través de los servidores de segundo nivel que hayan contratado. Para cada subdominio —como pasa con los dominios—, se debe tener toda la información sobre los equipos y servidores que le pertenecen.

A continuación, se muestra la estructura del árbol de nombres de dominio:

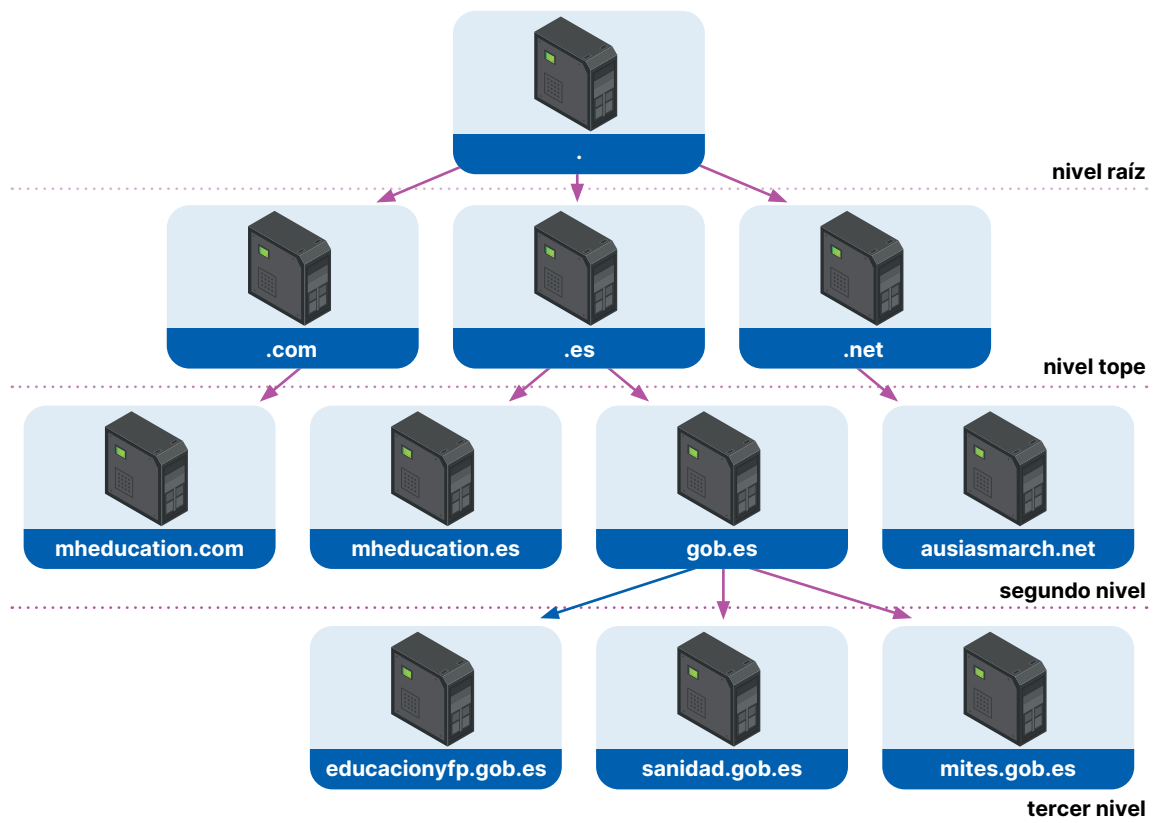


Figura 4. Estructura del árbol de nombres de dominio.

2.3. Tipos de dominio

Dentro del primer nivel, la ICANN tiene actualmente autorizados un gran número de dominios de primer nivel o TLD. Pero, en un principio, solo se establecieron dos tipos de TLD: los ccTLD específicos para cada país y los gTLD que son genéricos.

ccTLD (Country Code Top-Level Domains)

Dominios geográficos o de país. Se simbolizan con dos letras representativas del nombre del país. Los dominios de segundo nivel que pertenezcan a uno de estos dominios deben pertenecer al país correspondiente.

gTLD (Generic Top-Level Domains)

Dominios de primer nivel genéricos. Donde crearon 22 dominios de este tipo y todos tenían tres o más letras.

Esquema 3.

Existen varias subcategorías:



Esquema 4.

IMPORTANTE

El dominio genérico especial para gestionar la infraestructura es el .arpa, que sirve para realizar resoluciones inversas, es decir, a partir de una IP, averiguar el nombre de su dominio. En concreto, utilizaremos dos: **in-addr.arpa** para IPv4 e **ip6.arpa** para IPv6. Su funcionamiento se explica más adelante.

2.4. Resoluciones directas y resoluciones inversas

Existen dos tipos de resoluciones: iterativa y recursiva, pero dentro de estas podemos solicitar la información de dos maneras: directa o inversa.

1. Una resolución es **directa** cuando se obtiene la dirección IP a partir del nombre DNS.
2. Una resolución es **inversa** cuando se obtiene el nombre DNS a partir de la dirección IP.

La primera es la más utilizada, pero, en ciertas ocasiones, nos puede interesar utilizar la segunda. Veamos sobre la figura siguiente cómo funciona este segundo sistema de resolución sobre el árbol. Con él mostraremos desde el dominio raíz la estructura jerárquica de las zonas de resolución inversa. Del dominio raíz cuelgan dominios de primer nivel, entre ellos, el dominio .arpa, que es el especial de infraestructura para este tipo de resolución inversa. De este dominio nos interesan los dos dominios in-addr.arpa e ip6.arpa. Del dominio in-addr.arpa cuelgan servidores DNS para resolver direcciones IPv4 que comienzan por 1, 2 y, así, hasta 223. Del nodo 204 cuelgan, igual que antes, nodos con números comprendidos

entre 0 y 255. De uno de los nodos anteriores, el 74, cuelgan otra vez múltiples nodos, y se señala especialmente el nodo 99. Un recuadro que apunta a este nodo indica que el servidor correspondiente resuelve direcciones IP que comienzan por 204.74.99.z, donde z es el valor 100 para completar nuestra IP 204.74.99.100.

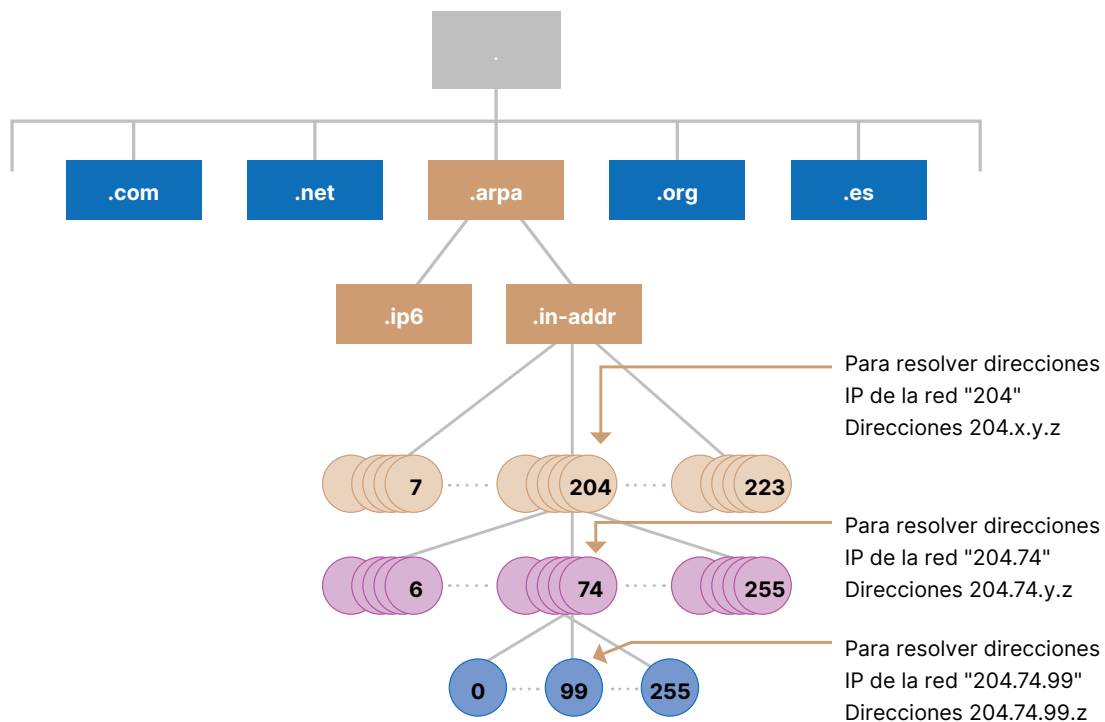


Figura 5. Estructura jerárquica de las zonas de resolución inversa.

2.5. Zonas primarias y secundarias

Si una zona se almacena en un servidor DNS, se dice que el servidor tiene autoridad o está autorizado sobre ella. Puede ocurrir que un mismo servidor DNS pueda tener autoridad sobre varias zonas. Esta es una parte del espacio de nombre de dominios sobre la que es responsable su servidor DNS, que puede tener autoridad sobre una o varias zonas.

La zona está constituida por varios registros. Cada registro sirve para resolver un nombre DNS o, en su caso, una dirección IP o cualquiera de los recursos que podemos tener en ella. También los registros tienen otras funciones, como indicar cuáles son los servidores DNS de la zona. A las zonas también se las llama zonas de autoridad o zonas de autoridad en el dominio.

El caso más habitual es que una zona resuelva los nombres de un dominio, es decir, que en un dominio haya solo una zona. Por esto se pueden confundir los términos 'zona' y 'dominio'. Una zona puede contener información sobre un dominio y sus subdominios. Nosotros administraremos cada zona en un archivo diferente para efectuar resoluciones.

Las zonas pueden ser de estos tipos de búsqueda o resolución:

» **Directa (forward):** dado un nombre DNS, nos devolverá su dirección IP asociada.

» **Inversa (reverse):** dada una dirección IP, nos devolverá su nombre DNS.

2.6. Tipos de servidores DNS

Existen varios tipos de servidores y un mismo servidor puede ser a la vez de varios tipos. Los servidores DNS pueden ser de cuatro tipos, dentro de dos categorías, y sus características son:

A. Servidores autoritarios o con autoridad:

» **Primario:** puede editar la zona o las zonas en las que actúa como servidor autoritario.

» **Secundario:** obtiene la información de la zona primaria de otro servidor primario por transferencia de zona. A pesar de no poder editarse una zona secundaria, los clientes DNS pueden usar estos servidores DNS sin ningún problema.

B. Servidores no autoritarios:

» **Caché:** puede editar la zona o las zonas en las que actúa como servidor autoritario.

» **Reenviador:** se considera así a un servidor DNS que ha sido designado por otro u otros servidores DNS para que se encargue de resolver nombres fuera del dominio en el que se encuentran.

Esquema 5.

2.7. Registro de recursos

Ahora nos toca llenar de contenido la zona primaria con los registros de recursos correspondientes. Cada registro de recursos (RR) tiene esta sintaxis (debajo se indica un ejemplo de datos):

<Propietario>	[<TTL>]	<Clase>	<Tipo>	<RDatos>
mcgrawDAW.lan.	0	IN	A	192.168.123.123

Tabla 2. Sintaxis de un registro de recursos.

En cada campo se utilizan los siguientes parámetros:

Propietario	Contiene un nombre FQDN para especificar que el nombre completo del propietario llevará un punto al final. Si no es completo, es decir, es un nombre relativo, entonces no termina en ese punto final. En este caso, se añadirá el nombre de la zona que representa con el carácter @. Si dejamos una cadena vacía, el propietario tomará el valor del registro que se haya declarado anteriormente.
TTL (<i>Time To Live</i>)	Marca el tiempo de vida del registro, que es un campo opcional. No se suele configurar, por lo que se deja vacío o con un valor 0, que indica lo mismo, como en nuestro ejemplo. Los servidores caché lo utilizan para saber cuándo tienen que actualizar los registros de las zonas descargadas y guardadas en su memoria. Dicho tiempo se puede expresar en días (d), horas (h), minutos (m) o segundos (s).
Clase	Indica la familia de protocolos para usar. Para nosotros siempre será "IN", ya que representa la familia de protocolos TCP/IP que se utiliza en Internet.
Tipo	Puede variar en función del campo Clase. En nuestro caso, para la clase IN en el siguiente subapartado de tipos de registros veremos cuáles están admitidos y para qué sirven.
RDatos	Es el registro con el dato por devolver en la consulta que nos planteen para este recurso.

Esquema 6.

A. Tipos de registros de recursos

Ahora debemos conocer la utilidad de cada tipo de registro para la clase IN, sabiendo escribir los registros correctamente para crear una zona y especificando lo que se resuelve en ella.

NOMBRE DE RECURSO	TIPO DE REGISTRO	FUNCIÓN
Inicio de autoridad (Start Of Authority)	SOA	Debe ser el primer registro de la zona. Identifica al servidor como autoritario sobre dicha zona y le da los parámetros de configuración.
Servidor de nombres	NS	Indica qué equipo ejerce como servidor de nombres en esta zona.
Dirección IPv4	A	Sirve para obtener su dirección IPv4 del nombre de dominio FQDN de un equipo que se ha solicitado resolver.
Dirección IPv6	AAAA	Igual que la A, pero para direcciones IPv6. Cada A representa 32 bits, es decir, 4 bytes. Las direcciones IPv6 son de 128 bits. Las especificaciones de las extensiones para soportar IPv6 en DNS están recogidas en el RFC 1886. Disponible desde BIND 9.
Puntero	PTR	Sirve para las búsquedas inversas, donde, dada una IP, nos devolverá el nombre de dominio FQDN.
Correo	MX	Permite saber qué máquinas van a gestionar la correspondencia de correo de nuestro dominio; puede haber varios e indica prioridades a la hora de configurarlos. El valor 0 es el prioritario.
Nombres canónicos	CNAME	Nos permite asignar alias a nombres reales de máquina. Muy utilizado para renombrar dichas máquinas con los servicios que atienden; por ejemplo, www para el servicio de web.
Texto	TXT	Nos permite almacenar cualquier tipo de información. Se suele usar para dejar la marca identificativa de nuestro dominio cuando usamos servicios externos como el correo electrónico de terceros.
Servicio	SRV	Indica la ubicación de los servidores para un determinado servicio.

Tabla 3. Tipos de registros para la clase IN y sus funciones.

Veamos un ejemplo de RR de tipo SOA:

<Propietario>	[<TTL>]	<Clase>	<Tipo>	<Registro de datos>
mcgrawDAW.lan.		IN	SOA	servidor1.mcgrawDAW.lan. (2022062901 ; número de serie 10800 ; actualización 3600 ; reintentos 604800 ; caducidad 86400 ; valor del TTL) ; final parámetros

Tabla 4. Ejemplo de tipo SOA.

Los valores de configuración están expresados en segundos, aunque también se pueden utilizar letras para representar las unidades de tiempo en semanas (*Weeks*), días (*Days*), horas (*Hours*) o minutos (*Minutes*). El anterior registro SOA equivaldría a (2022062901 3H 1H 1W 1D).



WEB

Para ampliar estos contenidos te recomendamos ver el vídeo «Lección 13: Seguridad en DNS (intypedia)» de la página de Criptored, que puedes encontrar en el siguiente enlace: <https://bit.ly/3lf5bb6>

03 Protocolo ligero de acceso a directorios (LDAP)

El servicio del protocolo ligero de acceso a directorios (LDAP, *Lightweight Directory Access Protocol*) es un protocolo de la capa de aplicación TCP/IP que utiliza la filosofía de cliente-servidor que nos va a permitir centralizar la autenticación y la autorización de usuarios de nuestros sistemas en uno o más servidores maestros o esclavos para ofrecer tolerancia a fallos, como tenemos en DNS.

El servicio LDAP sustituye al servicio NIS. Algunas de sus funciones más importantes son la autenticación de usuarios en aplicaciones web, sistemas operativos, encaminamiento y entrega de correo electrónico (SMTP, IMAP o POP), servicio de FTP e, incluso, la administración de zonas en servidores de nombres BIND 9, entre otras muchas.

3.1. LDAP

LDAP está basado en el protocolo X.500, surgido del modelo OSI, con la idea de tener un directorio que podríamos asemejar a una especie de guía telefónica o catálogo de una biblioteca de las organizaciones para que los administradores de sistemas puedan tener esta información centralizada. El directorio tiene información de objetos jerarquizada en categorías para compartir con el servicio que lo solicite. Funciona como una base de datos para almacenar los objetos que componen un directorio activo. Estos objetos pueden ser usuarios, grupos, unidades organizativas (UO) y recursos, como equipos clientes, servidores o impresoras, entre otros. Es un protocolo para gestionar el acceso de usuarios. En términos informáticos, un directorio es una base de datos especializada (también denominada repositorio de datos) que almacena información sobre objetos y que está específicamente diseñada para optimizar las operaciones de consulta. Un servicio de directorio es el software que almacena, organiza y facilita el acceso a la información de un directorio.

Por ejemplo, un directorio puede incluir información sobre los objetos (impresoras), que contendrá datos de tipo numérico (la velocidad en páginas por minuto) o cadena de caracteres (con su ubicación) o los flujos de impresión compatibles (por ejemplo, PostScript o ASCII), y así sucesivamente.

A. Funcionamiento

Uno o más servidores LDAP contienen los datos que conforman el árbol de directorio LDAP o base de datos troncal. El cliente LDAP se conecta con el servidor LDAP y le hace una consulta. El servidor contesta con la respuesta correspondiente, o bien con una indicación de dónde puede el cliente hallar más información. No importa con qué servidor LDAP se conecte el cliente, ya que siempre observará la misma vista del directorio: el nombre que se le presenta a un servidor LDAP hace referencia a la misma entrada a la que haría referencia en otro servidor LDAP. Esto aumenta la disponibilidad y la fiabilidad del directorio usando más de un servidor LDAP para mantener la información.

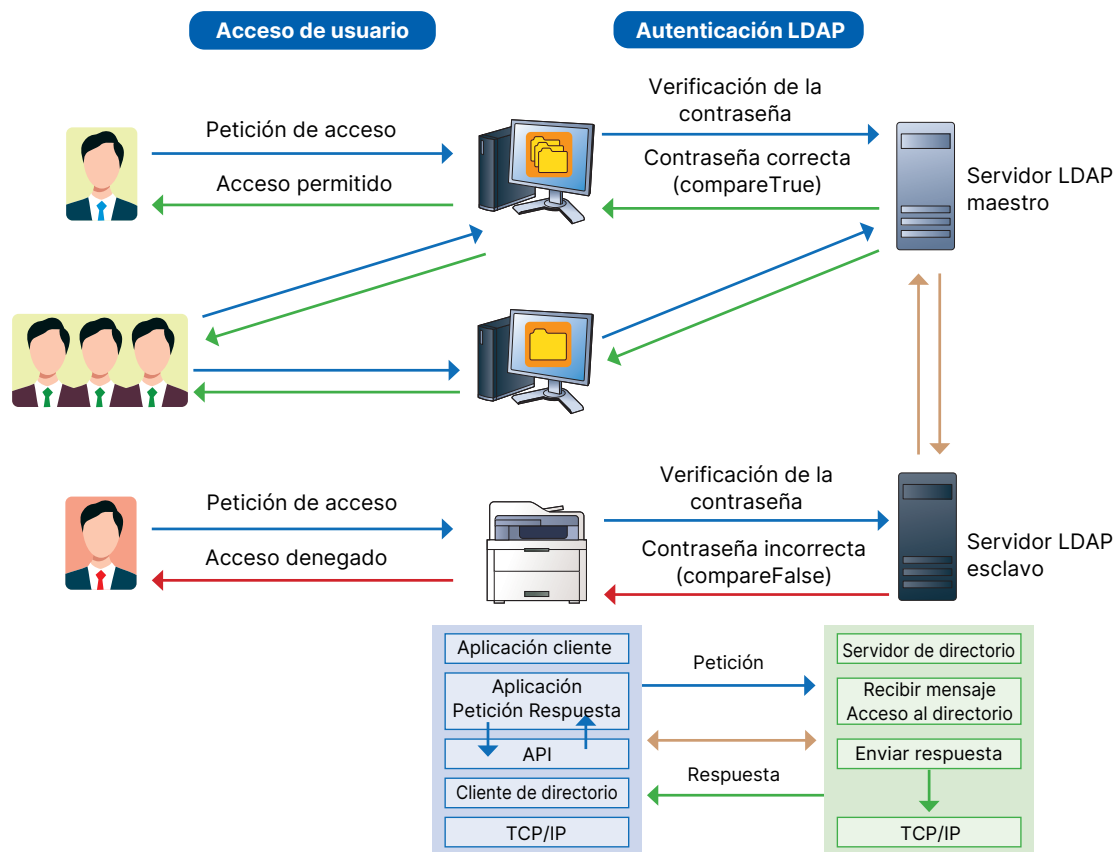


Figura 6. Funcionamiento de LDAP.

B. Esquemas de jerarquía

La información está estructurada en forma de árbol jerárquico invertido (como los directorios raíz del sistema de archivos Unix o el servicio DNS), según se observa en la figura siguiente. Este árbol se denomina árbol de información de directorio o DIT (*Directory Information Tree*), donde cada nodo se denomina entrada. Por su parte, cada entrada está formada por un conjunto de atributos, cada uno de los cuales es de un tipo y contiene uno o más valores.

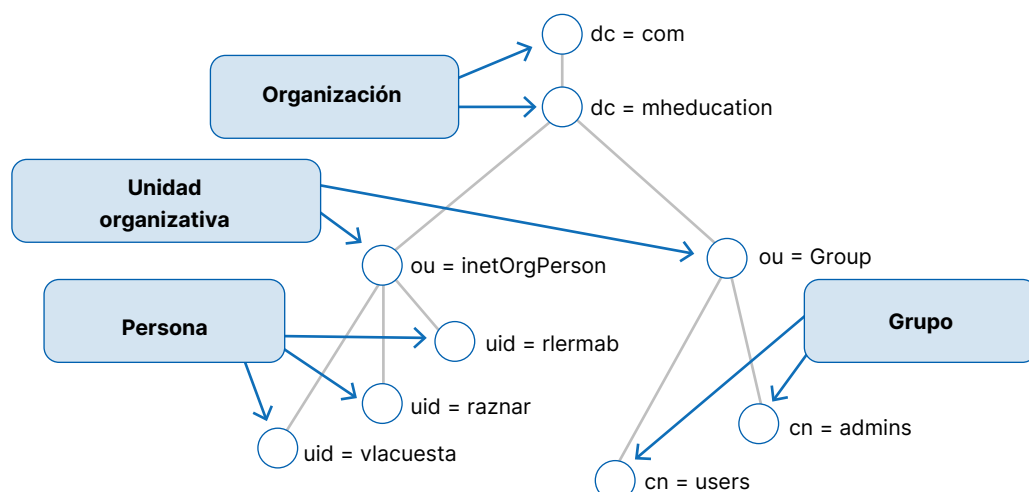


Figura 7. Árbol DIT.

C. Clases de objeto (*objectclass*)

Los atributos que contiene una entrada están condicionados por las clases de objeto a las cuales pertenece. Una clase de objeto agrupa conjuntos de atributos que pueden ser opcionales (indicado con MAY) u obligatorios (indicado con MUST) y la unión de todos los atributos de todas las clases de objeto de las cuales forma parte una entrada son los atributos permitidos por esta entrada.

La clase de objeto se reconoce con un identificador (OID) y, opcionalmente, el nombre (NAME). El atributo `objectClass` puede ser de tres tipos:

1

objectClass estructural: se utiliza para crear entradas (objetos de datos). Cada entrada debe tener solo una clase de este tipo.

2

objectClass auxiliar: se puede añadir a cualquier entrada. Por ejemplo, `dcObject` (que incluye el atributo `dc`, *domain component*, para hacer referencia a nombres DNS)

3

objectClass abstracta: utilizada para tratar entidades inexistentes. La clase de objeto abstracta más común es **top**, que constituye el nivel más alto de las jerarquías de objeto estructurales. Hay que tener en cuenta que una entrada solo puede tener una clase de objeto abstracta.

Esquema 7.

D. Herencia

Todas las clases de objetos tienen un origen común, la clase de objeto **top**. También es posible que una clase de objeto sea parte de una jerarquía. En este caso, hereda todas las características de las clases de objeto padre (clases superiores o SUP). En particular, hay que usar todos los atributos obligatorios de las clases superiores, además de los atributos adicionales necesarios que definen la nueva clase.

IMPORTANTE

Ejemplo de herencia en las clases de objeto. La clase de objeto **inetOrgPerson** deriva de la clase de objeto **organizationalPerson**, que, a su vez, deriva de **person**, que, al fin, tiene el padre **top**. Por lo tanto, **inetOrgPerson** incluirá los atributos de las clases de objeto **top**, **person**, **organizationalPerson** y las que ella misma haya definido.

Si una clase de objeto forma parte de una jerarquía, tiene que ser del mismo tipo (estructural o auxiliar) que el `objectClass` superior. La excepción a esta regla es si el superior es un tipo abstracto (por ejemplo, **top**).

E. Esquemas (schemas)

Los esquemas son unidades de embalaje; en general, todas las clases de objeto y atributos están definidas dentro de los esquemas. Se tiene que considerar que, inicialmente, el servidor de directorio no conoce ningún esquema, de forma que hay que cargar los archivos de esquema que contienen la descripción de las clases de objeto que el directorio ofrece. Los esquemas siguientes son de uso habitual:

- 1** | **core.schema:** para las clases y los atributos básicos.
- 2** | **cosine.schema:** para extensiones como se define en la RFC 1274, para identificación de usuario.
- 3** | **inetorgperson.schema:** cuando se necesitan más atributos, como se especifica en el RFC 2798.

Esquema 8.

Los esquemas son reutilizables. La creación de un objectClass dentro de un nuevo esquema puede usar atributos de objectClass definidos en otros esquemas. Dispondremos de algunas plantillas de esquemas predefinidos en el directorio `/etc/ldap/schema`.

F. Formato de intercambio de datos LDAP (LDIF)

Los servidores LDAP utilizan para insertar y extraer información del directorio el formato LDIF (*LDAP Data Interchange Format*), que está en formato de texto ASCII. Veamos su sintaxis y nuestro ejemplo:

Sintaxis del formato LDIF	Nuestro ejemplo
<pre>dn: <nombre distinguido> <nombre_ atributo>: <valor> # ... <nombre_atributo>: <valor> ... # Definir n objetos dn: <nombre distinguido> <nombre_ atributo>: <valor> # ... <nombre_atributo>: <valor></pre>	<pre>dn: uid=rlermab, ou=People, dc=mheducation, dc=com objectClass: inetOrgPerson cn: Raúl Vicent sn: Lerma-Blasco descripción: Docente mail: rlermab@mheducation.com</pre>

Esquema 9.

G. Ventajas

Un directorio LDAP aporta estas ventajas sobre los demás tipos de bases de datos:

- » Está optimizado para la lectura atómica de registros: se conecta, lee datos y cierra la conexión.
- » Replicar servidores distribuidos es muy sencillo y económico.
- » Es compatible con la mayoría de los servicios o aplicaciones a través de su API.
- » Unicidad: dispone de un modelo de nombres globales que asegura entradas únicas.
- » Usa un sistema jerárquico de almacenamiento de información equivalente a DNS.
- » Permite múltiples directorios independientes para diferentes servicios.
- » Forma parte de la familia de protocolos TCP/IP y admite seguridad con SSL.

3.2. OpenLDAP

Un servidor LDAP (*Lightweight Directory Access Protocol*) es un servidor de datos optimizado para las consultas rápidas de lectura y está orientado al almacenamiento de datos de usuarios a modo de directorio. Nosotros emplearemos una implementación de código abierto llamada OpenLDAP (otra implementación de LDAP es Active Directory de Microsoft).

OpenLDAP surge como una implementación libre de código abierto (*open source*) de LDAP. Es la alternativa para implementar estas características en sistemas basados en GNU/Linux, está publicado bajo su propia licencia y contiene todo lo necesario para crear un directorio activo en cualquiera de sus distribuciones: un servidor y diferentes utilidades o herramientas para los clientes. Nosotros utilizaremos Ubuntu Server 22.04 LTS, última versión de larga duración de 2022.

2 CASO PRÁCTICO

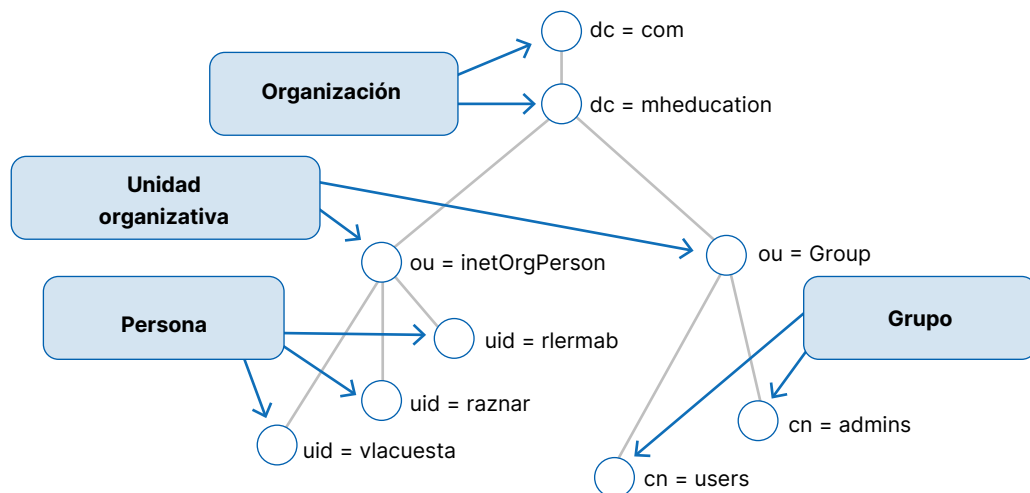


CONSTRUIR EL DIT DE TU CLASE

Dibuja el árbol jerarquizado de directorios LDAP de tu clase utilizando el esquema DNS de tu centro. Con que pongas a tres o cuatro compañeros o compañeras y a ti, será suficiente.

SOLUCIÓN

Tienes que obtener un dibujo parecido a este con los datos de tus compañeros y tu organización.



04 Despliegue con contenedores

Los contenedores son una tecnología disruptiva que ha llegado para quedarse y que está revolucionando todas las etapas de la vida del software. Nos ayudan a desarrollar y desplegar aplicaciones de una manera nunca antes vista. Son una metáfora de los contenedores de mercancías, que permiten aislar los productos que contienen, pues a nosotros nos van a permitir aislar nuestras aplicaciones del sistema y podremos interactuar con otros contenedores o con el resto de nuestra intranet o con Internet.

Los contenedores, en realidad, son equipos independientes con sus propias tarjetas de red internas, pero también externas. Podemos crear redes internas, como en los sistemas de máquinas virtuales, para poder enviar o recibir información, pero teniendo acceso al mundo exterior. Las etapas de desarrollo, pruebas y despliegue con contenedores se representan en la siguiente imagen.

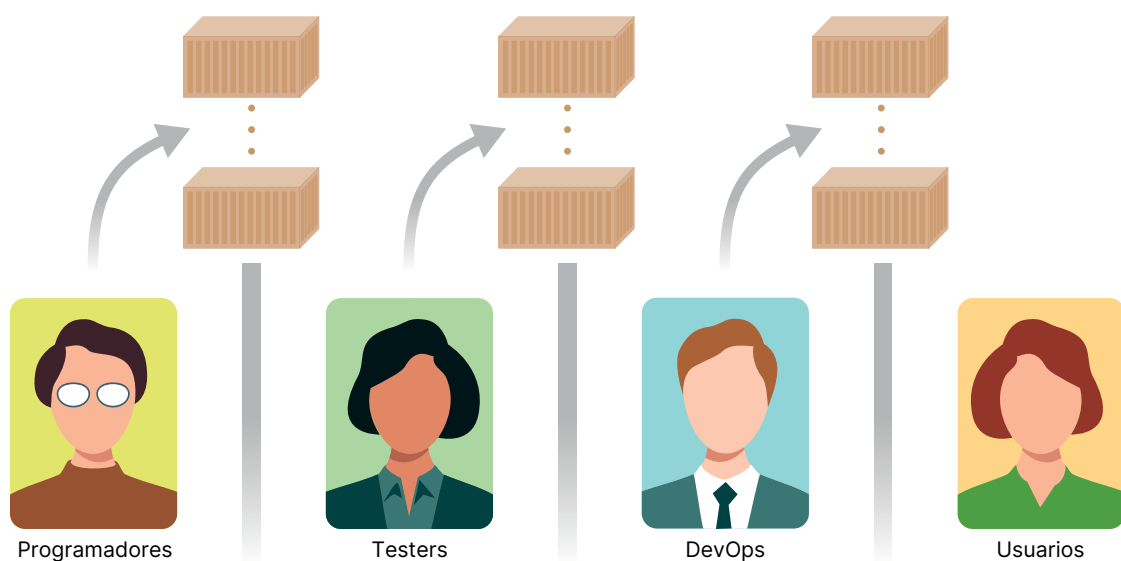


Figura 8. Etapas de desarrollo, pruebas y despliegue.

Para manejar más de un contenedor, han aparecido los orquestadores, que nos permiten sincronizar los diferentes contenedores que tengamos para que puedan verse y comunicarse entre ellos y trabajar conjuntamente como si de un único sistema integrado se tratara, pero con las ventajas de estar aislados.

Esta tecnología se ha integrado también en la computación en la nube, donde se ha creado una nueva capa denominada Contenedores como un servicio (*CaaS, container as a service*). Todos los proveedores de servicios en la nube, como AWS, Azure y Google Cloud, están compitiendo por dominar el mercado, si bien Docker es la plataforma más utilizada actualmente para el despliegue de aplicaciones web.

4.1. Contenedores

La virtualización de contenedores se basa en utilizar las funcionalidades del núcleo del sistema operativo GNU/Linux que le esté hospedando, de manera que funcione como un conjunto de uno o más procesos separados e independientes del resto. Partiendo de una imagen, vamos a poder montar capas sobre él para personalizarlo según nuestras necesidades en cada caso. Los contenedores son más rápidos y fáciles de desplegar al ocupar menos espacio, ya que, desde la fase de implementación hasta la de implantación, pasando antes por las de prueba, no cambiará nada dentro de ellos y podremos hacer un nuevo lanzamiento (*release*). Además, esta encapsulación aumenta la seguridad de todos los equipos y procesos que funcionan en las infraestructuras.

A. LXC

Los contenedores Linux Container (LXC) actuales existen desde 2008, pero hubo intentos anteriores de conseguir este objetivo.

La primera idea surgió en 1982, cuando prevalecían los sistemas Unix, mediante la creación de la orden **chroot**. Esta nos permitía cambiar el espacio de trabajo del superusuario (*root*), cambiando el directorio raíz para encapsularlo en un área de archivos más pequeña.

La siguiente idea es de 1999, cuando la distribución FreeBSD creó la orden **jail**, cuya idea era crear jaulas independientes que nos permitieran ejecutar tantos minisistemas como pudiéramos necesitar que ya compartían el núcleo de GNU/Linux.

Aunque el avance más importante se produjo a partir de 2001 en el proyecto VServer, donde también se hizo hincapié en que la seguridad informática estuviera muy presente, y requiere de las siguientes herramientas para poder aislarse:

1

Con la herramienta *cgroups*, vamos a poder gestionar la asignación de recursos a cada uno de los procesos. En el archivo **/etc/security/limits.conf** podemos definir límites.

2

La herramienta *namespaces* de GNU/Linux nos permite dar nombre a los procesos y a los recursos, de modo que, si tienen el mismo nombre, podrán verse, y permite que varios procesos o recursos también compartan el mismo nombre, de modo que podremos compartir recursos entre más de un proceso.

3

Y el demonio del sistema *systemd* permite administrar los espacios de nombres y sus procesos decidiendo cual inicializar o detener en cualquier momento.

Esquema 10.

Si unimos todas estas herramientas, vamos a tener contenedores enjaulados que, internamente, van a poder trabajar como superusuarios, pero, externamente, como un proceso más del equipo anfitrión, con los permisos y recursos muy limitados. A continuación, vemos el asistente de creación de contenedores LXC desde el panel de control de ProXmox VE.

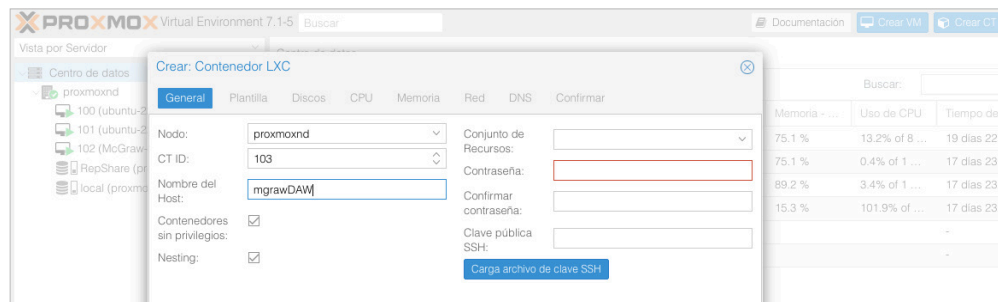


Figura 9. Asistente de creación de contenedores LXC.

En el sistema de virtualización ProXmox VE se pueden crear contenedores LXC directamente. Con la opción *Crear CT*, nos permite generarlos rápidamente a partir de plantillas que los propios fabricantes preparan para este tipo de nuevas máquinas en contenedores. Hoy existen evoluciones de estos, como LXD, pero el más famoso es Docker.

B. Docker

Creado por la empresa Docker Inc., es el producto privado de contenedores más utilizado. Ha creado una comunidad de usuarios y desarrolladores que están creando extensiones que se pueden integrar en su ecosistema y, así, aumentar el número de funcionalidades que se le pueden dar. Dispone de dos versiones:

1

La de código abierto es **Community Edition (CE)**. Esta versión contiene el núcleo del producto desarrollado a partir de los LXC. Es la que nosotros utilizaremos.

2

La de pago se llama **Enterprise Edition (EE)** e integra la versión CE; además, certifica sistemas y da soporte ininterrumpido a los clientes que lo contraten. Supone una garantía para las empresas que confían en ellos y requieren de ayuda.

Esquema 12.

Los dos elementos básicos (componentes) que utilizaremos son:

1

Imagen docker (*Docker Image*): es una plantilla especializada con un nombre y una versión separada con dos puntos para poder identificarlo. Por ejemplo, `ubuntu` o `alpine:latest`, que es la última versión disponible. Para tener siempre la misma versión, nos interesa indicar claramente cuál es la versión con la que va a funcionar siempre nuestro contenedor. Una imagen contiene instalada la aplicación que nos interese sobre un sistema operativo mínimo base. El concepto proviene de las imágenes de máquinas virtuales que, a su vez, proviene del concepto de ISO desde donde instalábamos sistemas operativos.

Por eso son de solo lectura. Si queremos actualizar una imagen, en realidad crearemos una nueva.

Podemos crear imágenes propias a partir de un archivo llamado **Dockerfile** y construyéndolas (*docker image build*). Sobre ellas, se les pueden aplicar nuevas capas u órdenes que implementen cambios sobre la imagen original y pueden hacerse tantas modificaciones como necesitemos (más adelante veremos las órdenes más importantes). Por tanto, las imágenes serán la manera de tener aplicaciones que luego ejecutaremos dentro de un contenedor.

2

Contenedor: es una instancia de una imagen puesta en funcionamiento —es decir, que está ejecutándose— que recibirá un identificador único de 64 caracteres. Podemos nombrarlo con los primeros caracteres que permitan conocer cuál es de los que tenemos en nuestro equipo. Recordemos que es como un miniordenador completo con tarjeta de red que funciona como un servicio de GNU/Linux; por tanto, podremos pararlo (*docker stop*), arrancarlo (*docker start*) o rearrancarlo (*docker restart*), siempre que lo hayamos construido y arrancado (*docker run*) previamente.

Esquema 11.

IMPORTANTE

Os recomendamos que uséis siempre que esté disponible la versión Alpine Linux, que está optimizada para funcionar en contenedores, ya que ocupa menos de 50 MB. Está disponible en su web *Alpine Linux > Downloads > VIRTUAL > Slimmed down kernel*. Optimizado para `x86_64`.

La estructura de funcionamiento (arquitectura) de Docker tiene varios elementos, que podemos ver también gráficamente en la figura siguiente:

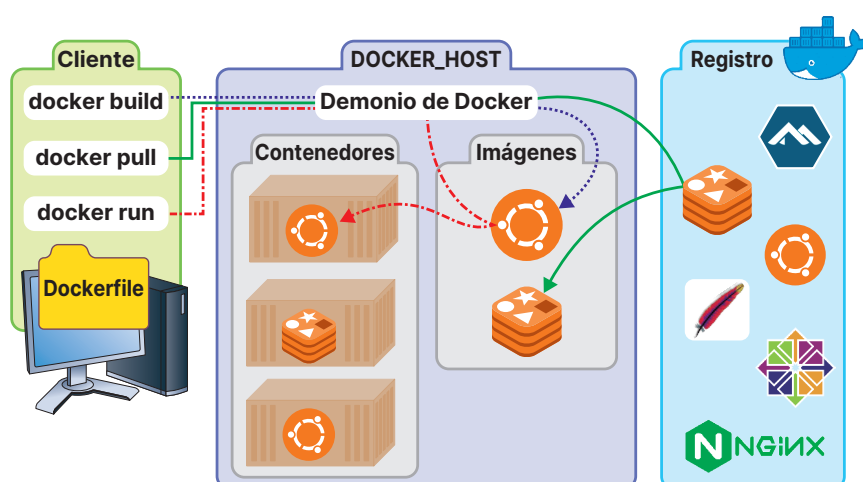


Figura 10. Arquitectura de Docker.

Registro (Registry) es un almacén de imágenes

La más conocida y utilizada es **Docker Hub**, desde donde podemos descargar imágenes públicas (*docker image pull*), pero también podemos enviar nuestras imágenes (*docker image push*) de manera privada en nuestra cuenta como hacemos con GitHub con nuestro código fuente. También existen otros registros de terceros.

1

El servidor (Docker Host)

Será el encargado de almacenar las imágenes que nos descarguemos de los registros y también será el encargado de cargar dichas imágenes en los contenedores que vayamos creando y poniendo en funcionamiento. Para ello, el demonio del Servicio Docker estará escuchando las órdenes que ejecutemos desde el cliente (*Docker CLI*).

2

El cliente (Docker CLI).

Es el canal de comunicación entre nosotros y el Docker Host. A modo de línea de comandos (*shell*), iremos ejecutando de modo interactivo lo que queramos que haga el servicio. O podemos indicarlo en un Dockerfile, a modo de *shell script*, para especificarle de qué imagen previa partiremos (*FROM*) y, luego, ya podremos indicarle el resto de las órdenes que aplicar para la construcción de la nueva imagen (*docker image build*), el contenedor (*docker build*) o de lo que queremos poner en ejecución (*docker run*).

3

Esquema 12.

Os recomendamos que os descarguéis solo imágenes docker oficiales y seguras de cuentas validadas que contengan la etiqueta DOCKER OFFICIAL IMAGE o de usuarios de confianza, ya que una imagen docker creada por terceros puede ser un vector de malware que puede infectar nuestro sistema o, incluso, toda nuestra organización.

4.2. Orquestadores

Dentro del mundo de los contenedores de software, un orquestador es un elemento más que nos va a permitir gestionar el funcionamiento de varios contenedores de manera más cómoda y centralizada. Cuando solo teníamos un contenedor con un único archivo Dockerfile, podíamos gestionarlo fácilmente, pero, cada vez que iba creciendo, también añadíamos más funcionalidades, lo que complicaba su gestión. O, si éramos meticulosos, podíamos ir cargando más contenedores especializados manualmente o crear redes Docker para que se pudieran comunicar entre ellas dentro del Docker Host. Pero tantas tareas repetitivas empezaban a ser tediosas y propensas a cometer errores, por lo que aparecieron los archivos en formato YAML con extensión `.yaml`, que se explicaron en el módulo de lenguaje de marcas, para poder indicar las órdenes por ejecutar en cada uno de los contenedores que necesitásemos.

A. Docker compose

Es el orquestador de la empresa Docker Inc. Nos permite gestionar el escalado local —más orientado a la parte de desarrollo— de más de un contenedor de manera centralizada con un único archivo **`docker-compose.yaml`**. Esta aplicación se tendrá que instalar en nuestro Docker CLI manualmente o automatizándolo con un *shell script* o un guion en PowerShell desde su cuenta de GitHub.

IMPORTANTE

La mayoría de los IDE de programación actuales, al igual que disponen de complementos para integrar el funcionamiento con GitHub, también disponen de complementos para Docker.

UAX FP